

13. ROS2 Distributed Communication

Note: The accompanying reference video uses the ROS Humble version, for reference only

1. Concept

Multi-machine communication, i.e., distributed communication, refers to a communication strategy that enables data exchange between different hosts over a network.

ROS2 itself is a distributed communication framework that conveniently facilitates communication between different devices. The middleware that ROS2 is based on is DDS. When on the same network, distributed communication can be achieved through DDS's domain ID mechanism (ROS_DOMAIN_ID). The general process is: before starting nodes, you can set the domain ID value. If different nodes have the same domain ID, they can discover and communicate freely with each other. Conversely, if domain ID values are different, communication cannot be achieved. By default, all nodes use domain ID 0 when starting. In other words, as long as you are on the same network, you don't need to do any configuration. Different nodes on different ROS2 devices can achieve distributed communication.

Distributed communication has a wide range of application scenarios, including autonomous vehicle formations, drone formations, remote control, etc. All these data exchanges depend on distributed communication.

2. Implementation

2.1. Default Implementation

You only need to have the master and slave machines [can be multiple] on the same network, and distributed communication is already achieved. For example, the master and slave machines are connected to the same WiFi or router.

Setting the virtual machine network in Windows to [Bridge Mode] puts it on the same network as the host.

Testing:

- Here we assume we have two hosts A and B. They can be any devices that can connect to the network, such as virtual machines, Raspberry Pi, Jetson, x86/ARM hosts, or single-board computers. You only need to have the same version of ROS2 installed.

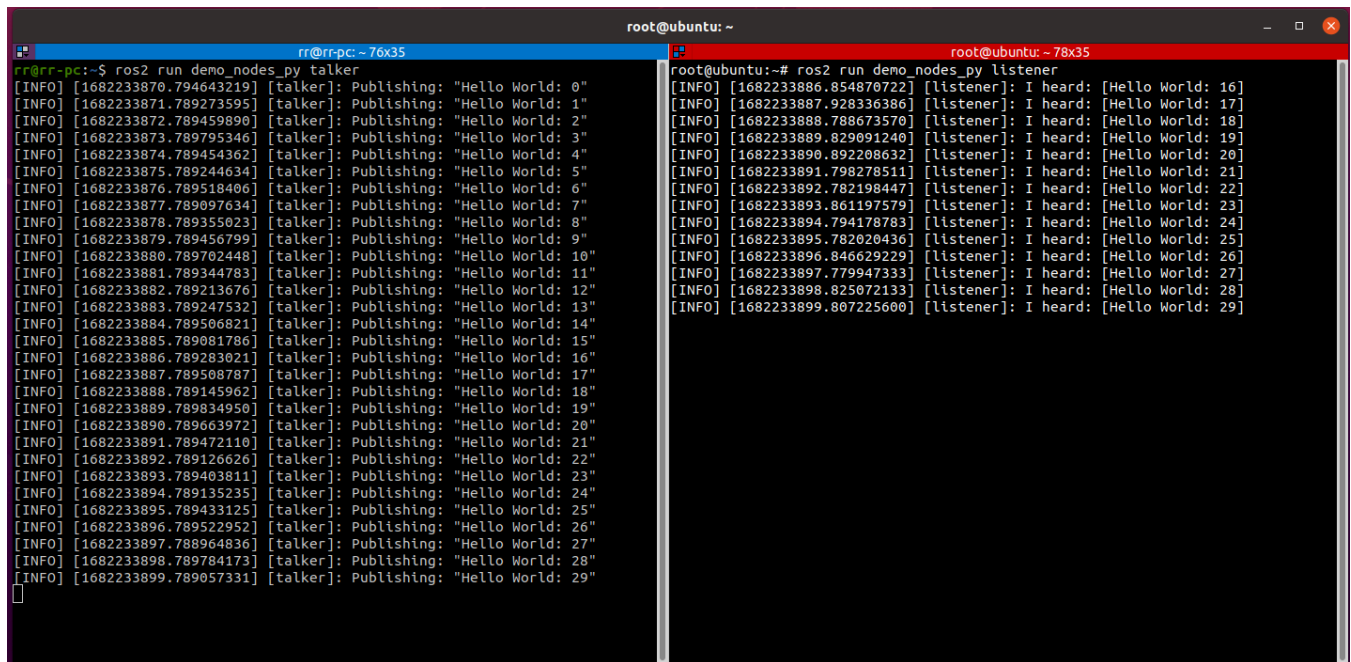
1. Execute on Host A:

```
ros2 run demo_nodes_py talker
```

2. Execute on Host B:

```
ros2 run demo_nodes_py listener
```

If the following is displayed: topics published by the host can be subscribed to by the slave in a timely manner, it means multi-machine communication has been achieved.



The image shows two terminal windows side-by-side. The left window is titled 'rr@rr-pc: ~ - 76x35' and shows the output of 'ros2 run demo_nodes_py talker'. It displays a series of log messages from '[INFO]' to '[INFO]' where a 'talker' node publishes 'Hello World' messages from index 0 to 29. The right window is titled 'root@ubuntu: ~ - 78x35' and shows the output of 'ros2 run demo_nodes_py listener'. It displays a series of log messages from '[INFO]' to '[INFO]' where a 'listener' node receives 'Hello World' messages from index 16 to 29. The messages in the right window are slightly delayed compared to the left window, indicating network latency.

2.2. Distributed Network Grouping

Assume you are in a network where other robots are also operating. To avoid interference from other robots, you can assign your robot to a group.

ROS2 provides a DOMAIN mechanism, similar to grouping. Only computers in the same DOMAIN can communicate. We can add this configuration to the .bashrc of both the host [mainboard] and the slave [virtual machine] to put them in the same group:

```
export ROS_DOMAIN_ID=<your_domain_id>
```

If the host [mainboard] and slave [virtual machine] are assigned different IDs, they cannot communicate with each other, achieving the purpose of grouping.

2.2.1 Example 1

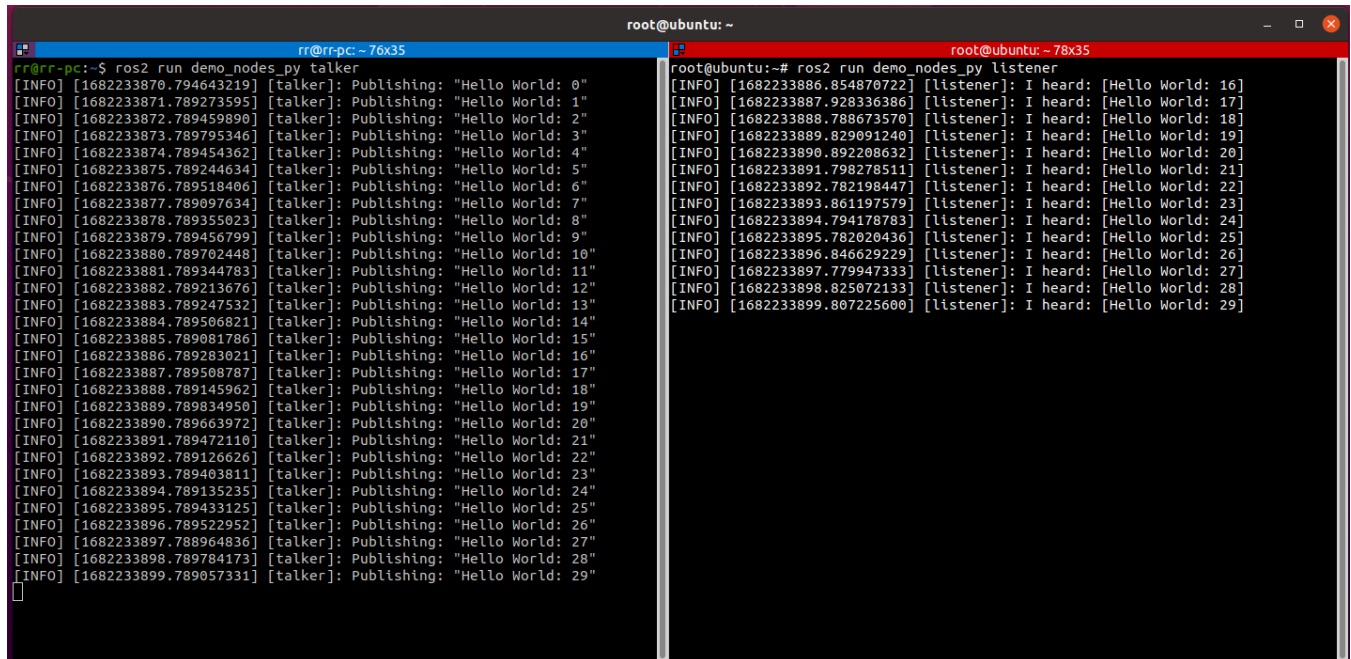
1. Execute on host [mainboard]:

```
echo "export ROS_DOMAIN_ID=6" >> ~/.bashrc # Here 6 is ROS_DOMAIN_ID. You don't have to use
6. As long as it follows ROS_DOMAIN_ID rules.
source ~/.bashrc
ros2 run demo_nodes_py talker
```

2. Simultaneously execute on slave [virtual machine]:

```
echo "export ROS_DOMAIN_ID=6" >> ~/.bashrc # keep this value consistent with the host
source ~/.bashrc
ros2 run demo_nodes_py listener
```

If the following is displayed: topics published by the host can be subscribed to by the slave in a timely manner, it means grouped multi-machine communication has been achieved.



The screenshot shows two terminal windows side-by-side. The left window, titled 'rr@rr-pc: ~ - 76x35', shows the output of 'ros2 run demo_nodes_py talker'. It displays a series of 'Hello World' messages from index 0 to 29, each preceded by an INFO log line with a timestamp and topic name. The right window, titled 'root@ubuntu: ~ - 78x35', shows the output of 'ros2 run demo_nodes_py listener'. It displays a series of 'I heard: [Hello World: X]' messages, where X corresponds to the index of the message received from the talker, also preceded by INFO log lines. This demonstrates successful communication between two machines in the same ROS2 domain.

2.2.2 Example 2

Control turtle motion through distributed communication:

- Run command on Host A:

```
ros2 run turtlesim turtlesim_node
```

- Run command on Host B:

```
ros2 run turtlesim turtle_teleop_key
```

3. Notes

When setting the ROS_DOMAIN_ID value, it's not arbitrary. There are some constraints:

1. It is recommended that ROS_DOMAIN_ID be in the range [0, 101], including 0 and 101;
2. The total number of nodes within each domain ID is limited to 120 or less;
3. If the domain ID is 101, the total number of nodes in that domain needs to be 54 or less.

4. DDS Domain ID Value Calculation Rules (Advanced Knowledge)

The domain ID value calculation rules are as follows:

1. DDS is based on TCP/IP or UDP/IP network communication protocols. Network communication requires specifying port numbers, which are represented by unsigned integers of 2 bytes, with a value range of [0, 65535];
2. Port allocation also follows rules and cannot be arbitrary. According to the DDS protocol, 7400 is used as the starting port, meaning usable ports are [7400, 65535]. Also, by default, each domain ID occupies 250 ports. Therefore, the number of domain IDs is: $(65535-7400)/250 = 232$ (count), with the corresponding value range being [0, 231];
3. The operating system also reserves some ports. When DDS uses ports, these reserved ports need to be avoided to prevent conflicts. Different operating systems have different reserved ports. The final result is: on Linux, usable domain IDs are [0, 101] and [215-231]. On Windows and Mac, usable domain IDs are [0, 166]. To be compatible across platforms, it is recommended to keep domain IDs in the range [0, 101];
4. Each domain ID occupies 250 ports by default, and each ROS2 node needs to occupy two ports. Additionally, according to the DDS protocol, within each domain ID's port range, the 1st and 2nd ports are Discovery Multicast port and User Multicast port. Starting from the 11th and 12th ports are the first node's Discovery Unicast port and User Unicast port within the domain. The ports occupied by subsequent nodes are sequentially incremented. Therefore, the maximum number of nodes in a domain ID is: $(250-10)/2 = 120$ (count);
5. Special case: When the domain ID value is 101, the latter half of its ports belong to the operating system's reserved ports, and the maximum number of nodes is 54.

You only need to be aware of the above calculation rules.